

CbcSolver: Class Architecture Overview

coin-or/Cbc — next branch — src/CbcSolver.hpp / src/CbcSolver.cpp

What is CbcSolver?

CbcSolver is the top-level driver class for the Cbc MIP solver executable and library entry point, encapsulating the entire solve pipeline — parameter initialization, command-queue parsing, LP relaxation solves, preprocessing, cut/heuristic configuration, branch-and-bound search, post-processing, and result reporting — that used to live in the free functions `CbcMain0/CbcMain1`, now thin backward-compatible wrappers delegating to `initialize()` and `run()`.

Historically this logic was not even a class method: it lived in the free function `CbcMain1` (paired with the smaller `CbcMain0`), a single ~10,000-line function built around one giant `switch` — still exactly this shape today on stable Cbc branches. A first refactor revived the class and moved `CbcMain1`'s body into it verbatim as `CbcSolver::run()` (`CbcMain0/CbcMain1` remain as thin wrappers). Further refactors (removing dead `#ifndef JJF_ONE / #if 0` blocks, splitting the branch-and-bound `case` into six cooperating private methods, dropping the legacy `-miplib/-unitTest` harness) then extracted coherent phases into the named methods below, reducing `run()` itself to a slim dispatcher. Local variables that needed to survive across phases were promoted to `private` data members (see §5). Most recently, three of the finer-grained **actions** used inside that pipeline — solving an LP relaxation, bound propagation, and clique strengthening — were promoted from private/inline/free-function code into public, individually-callable, solver-parameterized methods (see §2), so they can be driven directly on an arbitrary `OsiSolverInterface/OsiClpSolverInterface` instead of only on this `CbcSolver`'s own `model_.solver()` — e.g. from the C interface.

1 Public API surface

- **Construction & lifecycle:** `default / from-solver / from-model / copy` constructors, `operator=`, destructor (lines 2202–2735); `initialize()` sets up signal handlers and default parameters (replaces `CbcMain0`).
- **High-level solve:** `solve(file)`, `importModel(file)`, `solveLp(method)` — convenience one-call wrappers that chain `initialize()` + `run()` or solve the LP relaxation directly.
- **Result accessors:** `status()`, `objectiveValue()`, `bestBound()`, `bestSolution()`, `numCols()`, `nodeCount()`, `iterationCount()`, `hasSolution()` — valid after `solve()/run()` returns.
- **Run (replaces CbcMain1):** `run(argc,argv,...)` delegates to `run(deque,...)`, the primary command-queue-driven entry point and the root of the pipeline in §3.
- **Individual solve actions (§2):** `applyLpMethod(solver)`, `strengthenBounds(solver)`, `strengthenCliques(solver,...)` — the LP-relaxation solve, bound-propagation, and clique-strengthening actions used by the pipeline below, each independently callable and each accepting an optional target solver (defaulting to `model_.solver()`).
- **Heuristic / cut-generator configuration:** `configureHeuristics()`, `configureCutGenerators()` — public wrappers exposing internal BAB setup so callers can invoke it directly on a model without going through `run()`.
- **User extensions:** `addUserFunction()`, `setUserCallBack()`, `addCutGenerator()` register `CbcUser / CbcStopNow / CglCutGenerator` plugins.
- **Accessors:** `model()`, `babModel()`, `parameters()`, `intValue()` / `setIntValue()`, `doubleValue()` / `setDoubleValue()`, `originalSolver()`, `originalCoinModel()`, `getStatistics()`, and similar simple getters/setters over the state in §5.

2 Individual solve actions

Three actions that used to be entangled — inline, duplicated across call sites, or reachable only as a private/free-function helper on `model_.solver()` — are now public `CbcSolver` methods that each take an optional target solver parameter (`nullptr` defaults to `model_.solver()`), so any of them can be triggered directly on an externally-owned `OsiClpSolverInterface` while still honouring every LP/bound-propagation/clique option configured on this `CbcSolver` (`CbcParameters`, `ClpParameters`):

- **`applyLpMethod(OsiClpSolverInterface*targetSolver=nullptr, int forcedMethod=-1)`** — the unified root-LP-relaxation solver, and the *only* place that actually solves it: bound propagation ($\rightarrow$ `strengthenBounds()`) $\rightarrow$ clique merging “before” ($\rightarrow$ `strengthenCliques()`) $\rightarrow$ model-level LP settings (PSI positive-edge, objective scaling, vector mode) $\rightarrow$ LP racing (`-racingLP`) $\rightarrow$ full `ClpSolve` with every user-settable option (`-lpMethod` auto/dual/primal/barrier, presolve, crash/idiot/sprint, SLP, dualize, time limits). `forcedMethod` lets a caller pin the LP method (a `CbcParameters::LPMethod` value) for this call only, bypassing `-lpMethod=auto` and LP racing — used by the `DUALSIMPLEX/PRIMALSIMPLEX/ BARRIER` CLI commands, which used to hand-duplicate this entire option-setting sequence instead of calling it. Used internally by `solveInitialLp()` (root LP of the BAB pipeline) and, via `runSolveContinuous()`, by the `SOLVECONTINUOUS/DUALSIMPLEX/PRIMALSIMPLEX/ BARRIER` actions; exposed publicly so the exact same, fully-configured solve (with any forced method) can be driven on any solver instance.
- **`strengthenBounds(OsiSolverInterface*solver=nullptr)`** — runs the currently configured bound propagation (`-boundPropLevel`: singletons / fixpoint / MILP-based, up to `-boundPropMaxRounds` rounds via `CbcBoundPropagation`), falling back to legacy singleton-only tightening when propagation is off but `-singletonBounds` is on. Shared by `applyLpMethod()` (step 1) and the standalone `BOUNDPROP` action (`runBoundPropagation()`), which previously duplicated the same level-mapping and `CbcBoundPropagation::run()` call inline.
- **`strengthenCliques(solver, mode, strengthenMode, ...)`** — (re)builds the conflict graph of the target solver and strengthens set-packing/partitioning cliques against it (extend/dominate), resolving the LP afterwards only in “after” mode. Thin `CbcSolver`-level wrapper filling in this solver's own `model/handler/mipStart_` context around the lower-level free function that does the actual conflict-graph work; used by `applyLpMethod()`'s “before” step and by `babPostPreprocessCleanup()`'s “after” step, replacing what used to be two independent call sites each re-deriving the same handler/model/mipStart arguments.

As part of exposing these, a dead-code duplicate was removed from `solveInitialLp()`: a second clique-strengthening pass and a second `model_.initialSolve()` call, both gated on `lpMethodResult > 0` — a condition that can never hold, since `applyLpMethod()` only ever returns `-1` or `0` and already performs both the clique pass and the LP solve internally.

3 The branch-and-bound pipeline

`run()` dispatches on the current `CbcParam` command. Besides simple one-shot actions (§4), the `BAB` command drives the solver's core pipeline,

implemented as six cooperating private methods called in sequence (see the full diagram on the next page):

1. `babSetupAndRootLp()` — cutoff sign flip, legacy `OsiSolverLink` path, solves the root LP via `solveInitialLp()` → `applyLpMethod()`.
2. `babConfigureBabModel()` — (re)builds `babModel_` from `model_`, configures the conflict-graph branching ranker (`RankerConfig`), tunes factorization frequency.
3. `preprocess()` — runs `CglPreProcess` when preprocessing is enabled.
4. `babPostPreprocessCleanup()` — cgraph/cliue-strengthening cleanup (via `strengthenCliques()`), post-preprocess bound tightening.
5. `babConfigureSearchModel()` — knapsack expansion, cost-based priorities, and invokes `configureHeuristics()` / `configureCutGenerators()`.
6. `babExecuteSearchAndPostprocess()` — SOS/priority setup, the actual `babModel_` → `branchAndBound()` call, search-statistics bookkeeping, and finally `postprocess()` to translate the solution back and report results.

Each phase returns a status code consumed by `run()` to decide whether to continue, `break` the BAB case, or `return` from `run()` outright — preserving the exact control flow of the original monolithic function.

4 Other `run()` actions

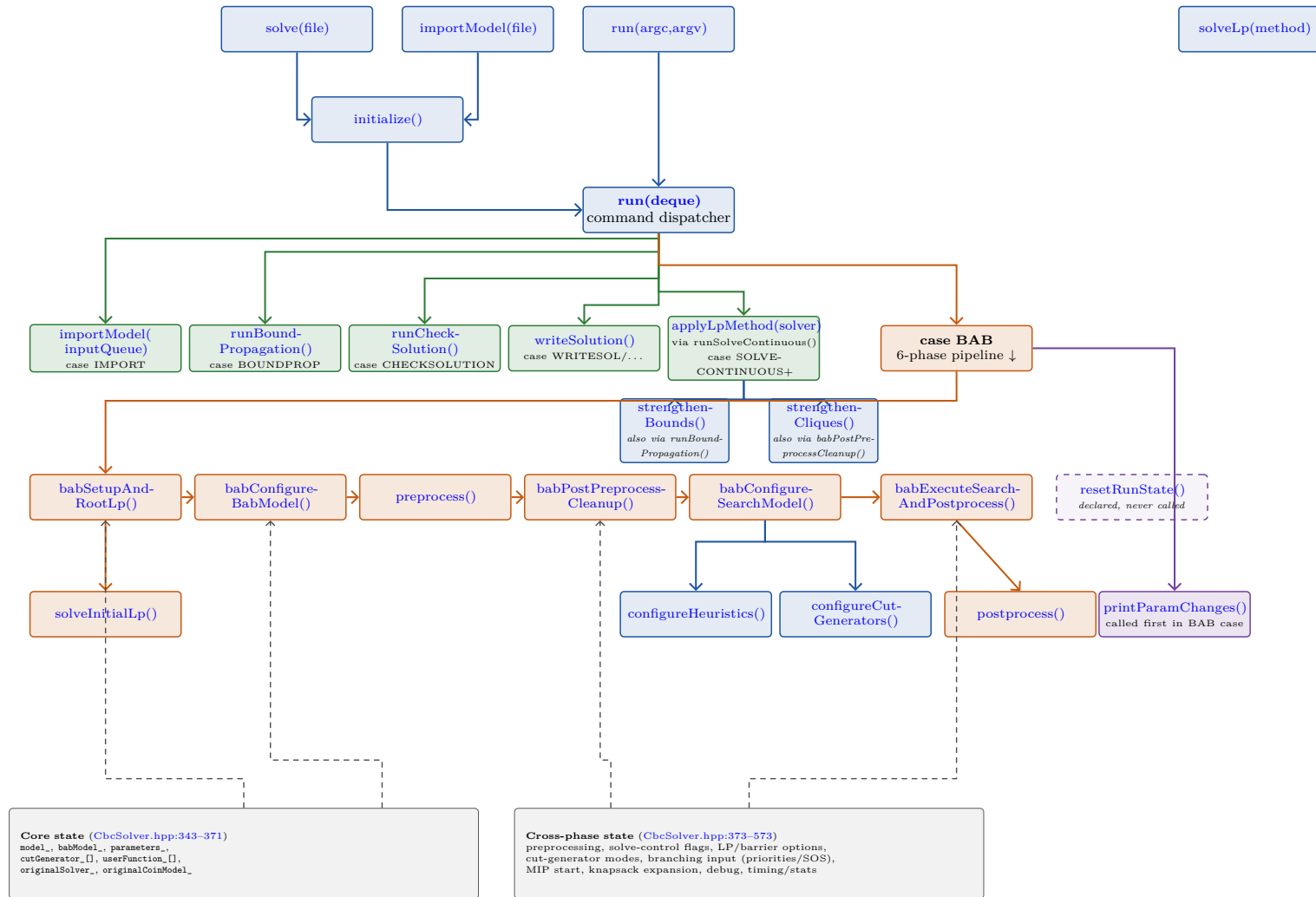
- **IMPORT:** private `importModel(inputQueue,...)` reads an MPS/LP/GMPL file.
- **SOLVECONTINUOUS / DUALSIMPLEX / PRIMALSIMPLEX / BARRIER:** private `runSolveContinuous(forcedMethod,...)` sets up the shared LP progress handler, calls `applyLpMethod()` (forcing dual/primal/barrier for the latter three commands), maps the resulting Clp status onto `model_/babModel_`, and populates the AMPL info buffer. These four commands used to each keep an independent ~200–450-line copy of this sequence; the CLP-method commands additionally hand-duplicated `applyLpMethod()`'s entire `ClpSolve` option-setting logic.
- **BOUNDPROP:** `runBoundPropagation()` runs standalone bound propagation (thin CLI wrapper around `strengthenBounds()`).
- **CHECKSOLUTION:** `runCheckSolution()` writes a solution-validation report.
- **WRITESOL / PRINTSOL / ...:** `writeSolution()` writes solutions in various formats.
- **Bookkeeping:** `printParamChanges()` prints a parameter-change banner at the top of the BAB case; `resetRunState()` is declared/defined but currently *never called* (orphaned — dashed outline in the diagram).

5 State organization

`Private data` falls into two groups. **Core state** (`model_`, `babModel_`, `parameters_`, `cutGenerator_`, `userFunction_`, ...) existed in the original class. **Cross-phase state** (lines 373–573) — preprocessing (`process_`, `saveSolver_`), solve-control flags, LP/barrier options, cut-generator modes, branching input (priorities/SOS/pseudo-costs), MIP start, knapsack-expansion buffers, debug, lot-sizing, and timing/statistics — used to be *local variables* inside the monolithic `run()`. Promoting them to member fields is what let `run()` be split into the cooperating methods of §3: each extracted method reads/writes the subset of this state it needs directly, instead of receiving dozens of by-reference parameters.

Colour legend used on the diagram (next page; node fill and arrow colour share the same code, so each call arrow is coloured like the method it leaves): ■ public API ■ BAB pipeline (§3) ■ other `run()` actions ■ bookkeeping helpers ■ state / data structures (dashed = data reads/writes)

CbcSolver — method call graph & state ownership (node labels are hyperlinks to `coin-or/Cbc@next`)



Arrows show call/return relationships within the modularized BAB pipeline (§3); dashed grey arrows indicate representative reads/writes of the shared state panel on the left (most pipeline methods touch several state groups — only the busiest links are drawn to keep the diagram readable). All labels link to the corresponding line in `coin-or/Cbc`, branch `next`.